

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Artificial Intelligence and Data Science

### ASSESSMENT TOOL 1 – Experiments

|                  |                                                                                                                                |               |                                 |
|------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------|---------------------------------|
| Course Code:     | DSA0307                                                                                                                        | Course Title: | Natural Language Processing     |
| CO Assessed:     | CO3 – Precision                                                                                                                | Assessment:   | Assessment Tool 1 – Experiments |
| Weightage:       | 50% of CIA                                                                                                                     |               |                                 |
| CO5 Statement:   | Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3) |               |                                 |
| Student Name:    | T Deekshith                                                                                                                    | Reg. No.:     | 192472253                       |
| Section / Batch: | 2024                                                                                                                           | Date:         | 06-08-2026                      |

#### Code of Conduct

*I T Deekshith(192472253) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Deekshith

#### Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo\_Name\_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
  - GitHub Repository Link in GCR
  - Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

| Section / Topic                                                                                                                                 | Focus                                                                                                                                               | Experiment Questions |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing | Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.              | Q1                   |
| Agreement, Feature Structures, Sub Categorization                                                                                               | Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules. | Q2                   |
| Probabilistic Context-Free Grammars (PCFGs)                                                                                                     | Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.                         | Q3                   |

### Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

#### Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

#### Grammar:

```
S → NP VP
NP → Det N
VP → V NP

Det → the | a
N → student | teacher | book
V → reads | likes
```

#### Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

#### Function Prototype

```
def parse_cfg(sentence):
    pass
```

#### Test Cases

| Input                    | Expected Output  |
|--------------------------|------------------|
| the student reads a book | Valid Parse Tree |
| a teacher likes the book | Valid Parse Tree |
| student reads book       | Invalid Sentence |
| the book likes a teacher | Valid Parse Tree |
| reads the student book   | Invalid Sentence |

### Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

#### Aim

To develop a Python program to validate whether a sentence follows the given CFG rules and construct its parse tree.

#### Algorithm

1. Define the grammar vocabulary (Determiners, Nouns, and Verbs).
2. Read the sentence from the user.
3. Split the sentence into words.
4. Check whether the sentence follows the pattern:  
**Det N V Det N**
5. If valid, print the parse tree.
6. Otherwise, print "Invalid Sentence".

## Program

```
def parse_cfg(sentence):
    words = sentence.lower().split()
    det = ["the", "a"]
    noun = ["student", "teacher", "book"]
    verb = ["reads", "likes"]
    if len(words) != 5:
        print("Invalid Sentence")
        return
    if (words[0] in det and
        words[1] in noun and
        words[2] in verb and
        words[3] in det and
        words[4] in noun):
        print("Valid Parse Tree\n")
        print("      S")
        print("    /  \")
        print("  NP   VP")
        print(" / \  / \")
        print("Det N V NP")
        print("    /  \")
        print("  Det  N")
        print("\nLeaves:")
        print(words)
    else:
        print("Invalid Sentence")

sentence = input("Enter Sentence: ")
parse_cfg(sentence)
```

## Sample Output

Enter Sentence:

the student reads a book

Valid Parse Tree

```
      S
    /  \
  NP   VP
 / \  / \
Det N V NP
    /  \
  Det  N
```

Leaves:

['the', 'student', 'reads', 'a', 'book']

## Result

Thus, the sentence was successfully validated using CFG production rules and its parse tree was generated.

## Experiment 2: Agreement and Feature Structure Checking

### Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

### Feature Rules

he → singular, third person  
she → singular, third person  
it → singular, third person  
they → plural

runs → singular verb  
writes → singular verb

run → plural verb  
write → plural verb

### Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

### Function Prototype

```
def check_agreement(subject, verb):  
    pass
```

### Test Cases

#### Subject Verb Expected

|      |        |       |
|------|--------|-------|
| he   | runs   | True  |
| he   | run    | False |
| they | run    | True  |
| they | writes | False |
| she  | writes | True  |

## Experiment 2: Agreement and Feature Structure Checking

### Aim

To verify subject–verb agreement using feature structures.

### Algorithm

1. Store singular and plural subject rules.
2. Store singular and plural verb rules.
3. Read subject and verb.
4. Compare their grammatical features.
5. Print True if both match.
6. Otherwise print False.

### Program

```
subjects = {
    "he":"singular",
    "she":"singular",
    "it":"singular",
    "they":"plural"
}
verbs = {
    "runs":"singular",
    "writes":"singular",
    "run":"plural",
    "write":"plural"
}

def check_agreement(subject, verb):

    if subject not in subjects or verb not in verbs:
        return False

    return subjects[subject] == verbs[verb]

subject = input("Enter Subject: ").lower()
verb = input("Enter Verb: ").lower()

print(check_agreement(subject, verb))
```

### Sample Output

Enter Subject:

he

Enter Verb:

runs

True

Another Output

Enter Subject:

they

Enter Verb:

writes

False

## Result

Thus, subject–verb agreement was successfully verified using feature structures.

## Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

### Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

### Grammar

$S \rightarrow NP VP$  [1.0]

$NP \rightarrow \text{John}$  [0.6]

$NP \rightarrow \text{Mary}$  [0.4]

$VP \rightarrow \text{runs}$  [0.5]

$VP \rightarrow \text{walks}$  [0.5]

### Probability Formula

$P(\text{Sentence})$

=

$P(S \rightarrow NP VP)$

$\times P(NP)$

$\times P(VP)$

### Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

### Function Prototype

```
def sentence_probability(sentence):
```

```
    pass
```

### Test Cases

| Input Sentence | Expected Probability |
|----------------|----------------------|
|----------------|----------------------|

|           |      |
|-----------|------|
| John runs | 0.30 |
|-----------|------|

|            |      |
|------------|------|
| John walks | 0.30 |
|------------|------|

|           |      |
|-----------|------|
| Mary runs | 0.20 |
|-----------|------|

|            |      |
|------------|------|
| Mary walks | 0.20 |
|------------|------|

|            |      |
|------------|------|
| Peter runs | 0.00 |
|------------|------|

### Experiment 3: Probabilistic Context-Free Grammar (PCFG)

#### Aim

To compute sentence probability using Probabilistic Context-Free Grammar.

#### Algorithm

1. Store grammar probabilities.
2. Read a two-word sentence.
3. Check whether NP and VP exist.
4. Compute

$$P = P(S \rightarrow NP VP) \times P(NP) \times P(VP)$$

5. Display probability.
6. If sentence cannot be generated, return 0.

#### Program

```
np_prob = {
    "John":0.6,
    "Mary":0.4
}

vp_prob = {
    "runs":0.5,
    "walks":0.5
}

def sentence_probability(sentence):

    words = sentence.split()

    if len(words)!=2:
        return 0

    np = words[0]
    vp = words[1]

    if np not in np_prob or vp not in vp_prob:
        return 0

    return 1.0 * np_prob[np] * vp_prob[vp]

sentence = input("Enter Sentence: ")

print("Probability =", sentence_probability(sentence))
```

## Sample Output

Enter Sentence:

John runs

Probability = 0.3

Another Output

Enter Sentence:

Mary walks

Probability = 0.2

Another Output

Enter Sentence:

Peter runs

Probability = 0

## Result

Thus, the sentence probability was successfully computed using Probabilistic Context-Free Grammar (PCFG).

## Rubrics for Calculation

| Criteria                  | Weightage | Level 4 – Exemplary                                                                           | Level 3 – Proficient                                        | Level 2 – Developing                                | Level 1 – Beginning                              |
|---------------------------|-----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25        | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30        | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20        | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15        | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10        | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |



| Q. No. /<br>Criteria<br>Level | Understanding<br>of Concepts | Experimental<br>Design | Use of<br>Tools | Analysis<br>of Results | Document<br>ation | Sub. Total | Total<br>% |
|-------------------------------|------------------------------|------------------------|-----------------|------------------------|-------------------|------------|------------|
| 1                             |                              |                        |                 |                        |                   |            |            |
| 2                             |                              |                        |                 |                        |                   |            |            |
| 3                             |                              |                        |                 |                        |                   |            |            |

| Faculty In-charge | Course Coordinator | Program Director |
|-------------------|--------------------|------------------|
|                   |                    |                  |
| Signature: _____  | Signature: _____   | Signature: _____ |
| Date: _____       | Date: _____        | Date: _____      |